

Tema 4: Búsqueda No Informada

Razonamiento y Planificación Automática — UNIR | Guía de Estudio para Evaluación Final

1. ¿Qué es la Búsqueda en IA?

Definición: Búsqueda en Inteligencia Artificial

En IA, "búsqueda" no significa buscar literalmente — es un mecanismo para **resolver problemas**. Consiste en encontrar una secuencia de acciones (un plan) que lleve al agente desde su estado actual hasta un estado meta.

El agente opera sobre un **modelo simbólico del mundo** representado como un grafo, no sobre el mundo directamente. "Buscar" es encontrar el mejor camino en ese grafo desde el nodo inicial hasta un nodo meta.

Correspondencia Mundo – Modelo – Grafo

Mundo Real	Modelo del Agente	Representación en Grafo
Situación	Estado	Nodo
Situación actual	Estado inicial	Nodo inicio
Acciones	Operadores	Arcos
Secuencia de acciones	Plan	Camino

💡 **Pregunta clave de sesión:** ¿Por qué no simplemente aplicamos Dijkstra o un algoritmo de camino mínimo clásico?

Respuesta: Porque el grafo *no lo conocemos*. El agente tiene que explorar el grafo desde cero y construirse una idea del mundo a medida que avanza. De ahí el nombre: *búsqueda no informada*.

2. Clasificación de los Problemas de Búsqueda

2.1 Por uso de heurística

Tipo	Heurística	Descripción
Búsqueda no informada	✗ No usa	Evalúa estados sin saber si son mejores o peores que el anterior. Sin guía.
Búsqueda informada	✓ Sí usa	Emplea una función heurística para guiar la búsqueda hacia soluciones óptimas.

Función Heurística

En IA, una función heurística es una *estimación de lo que falta para conseguir el objetivo*. No es exacta, pero sirve para guiar la búsqueda inteligentemente.

2.2 Por tipo de agente

Tipo	Cuándo busca	Cuándo actúa
Offline (deliberativo)	Antes de actuar — encuentra el plan completo primero	Una vez terminada la búsqueda
Online (reactivo)	Mientras actúa — búsqueda a corto plazo	Durante la búsqueda, con riesgo de error

3. Mecanismos para Resolver Problemas de Búsqueda

Mecanismo	Descripción	Limitación principal
Tablas de actuación	Para cada situación hay una entrada en una tabla con la secuencia de acciones	Grave problema de escalabilidad y memoria
Algoritmos específicos del dominio	El diseñador codifica un método para un dominio concreto	Solo funciona para ese dominio; no generaliza

Métodos independientes del dominio	Usan un modelo simbólico + algoritmo de búsqueda genérico	Mayor flexibilidad; no requiere conocer la solución previamente
---	---	---

⚠ Los algoritmos de búsqueda no informada son métodos independientes del dominio. El mismo algoritmo puede usarse para el puzzle-8, las Torres de Hanoi o cualquier otro problema — solo cambia la definición del espacio de estados.

4. El Espacio de Estados y el Algoritmo General

4.1 Conocimiento a priori del agente

El agente no conoce el grafo completo, pero sí dispone de esta información desde el inicio:

Notación	Significado
s_0	Estado inicial
$\text{expandir}(s) = \{s_1, \dots, s_n\}$	Conjunto de estados sucesores del estado s
$\text{meta}(s) : \text{verdad} \mid \text{falso}$	Función de evaluación de s como estado meta
$c(s_i, s_j) : v, v \in \mathbb{R}$	Coste de aplicar el operador que lleva de s_i a s_j
$c(s_1 s_2 \dots s_n) = \sum_{k=1}^{n-1} c(s_k, s_{k+1})$	Coste del plan completo

4.2 Algoritmo General de Búsqueda

Input: Estado inicial S_0 , Estado final G

```
1: colaAbierta ← {S0}
2: mientras colaAbierta ≠ ∅
3:     nodo ← extraer primero de colaAbierta
4:     si meta(nodo) entonces
5:         retornar camino a nodo
6:     fin si
7:     sucesores ← expandir(nodo)
8:     para cada sucesor ∈ sucesores hacer
9:         sucesor.padre ← nodo
10:        colaAbierta ← colaAbierta ∪ sucesor
11:    fin para
12: fin mientras
13: retornar plan vacío o problema sin solución
```

Clave: Los tres algoritmos (BFS, DFS, UCS) son variantes de este algoritmo general. La diferencia entre ellos está en **una sola cosa: el orden en que insertan los nuevos nodos en la cola abierta.**

4.3 Características para evaluar un algoritmo

Característica	Pregunta que responde
Compleitud	¿Encuentra la solución si existe?
Optimalidad	¿Si hay varias soluciones, encuentra la mejor (menor coste)?
Complejidad en tiempo	¿Cuánto tarda en encontrar la solución?
Complejidad en espacio	¿Cuánta memoria necesita para encontrar la solución?

Notación $O()$ — "O grande"

Expresa cómo *crece* el coste de un algoritmo en función del tamaño del problema. No dice el tiempo exacto, sino el orden de crecimiento.

- $O(n)$ — crece linealmente
- $O(n^2)$ — crece cuadráticamente
- $O(b^d)$ — crece **exponencialmente** (el más común en búsqueda)

Donde **b** = factor de ramificación (hijos por nodo) y **d** = profundidad de la solución.

💡 ¿Qué es el factor de ramificación b?

Es el número promedio de sucesores que genera cada nodo al expandirse. Si $b = 3$, cada nodo tiene 3 hijos. El número de nodos por nivel crece exponencialmente: nivel 1 $\rightarrow 3$,

nivel 2 $\rightarrow 9$, nivel 3 $\rightarrow 27$... en general: b^d .

5. Búsqueda en Amplitud — BFS (Breadth-First Search)

Definición

Estrategia que genera el árbol de búsqueda **nivel por nivel**. Expande todos los nodos del nivel i antes de pasar al nivel $i + 1$. Considera primero caminos de longitud 1, luego de longitud 2, etc.

Mecánica

La cola abierta funciona como una **FIFO** (First In, First Out): los nuevos sucesores se insertan **al final** y se extrae siempre **del inicio**. Así se garantiza que siempre se expanden primero los nodos más antiguos (menos profundos).

Analogía: Es como inundar un terreno con agua — el agua se expande uniformemente en todas direcciones al mismo tiempo, nivel por nivel. La solución más cercana al origen siempre se encuentra primero.

Propiedades

Característica	Resultado	Justificación
Compleitud	✓ Sí	Explora todos los nodos por nivel; si la solución existe, la encuentra
Optimalidad	✓ Sí (solo si $\text{coste} = 1$)	Al explorar nivel por nivel, el primer nodo meta encontrado tiene la menor profundidad
Complejidad en tiempo	✗ $O(b^d)$	Exponencial en la profundidad de la solución
Complejidad en espacio		Debe guardar todos los nodos del nivel actual en memoria



$$O(b^d)$$

⚠ Problema de escalabilidad

Con $b = 10$ y $d = 12$: 10^{12} nodos $\rightarrow$ **1.000 TB de memoria**. Con $d = 14$: 3,5 años de tiempo de cómputo. BFS es correcta en teoría pero inviable en problemas profundos.

💡 Pregunta de sesión: ¿BFS es óptima siempre?

Respuesta: No. Solo es óptima cuando todos los costes son iguales ($= 1$). Si los costes varían, un camino de menos pasos puede ser más caro que uno de más pasos $\rightarrow$ BFS falla. Para ese caso existe UCS.

6. Búsqueda en Profundidad — DFS (Depth-First Search)

Definición

Estrategia que explora el árbol yendo **tan profundo como sea posible** por una rama antes de retroceder. Cuando llega a un nodo sin sucesores, hace **backtracking** al nodo anterior y prueba otra rama.

Mecánica

La cola abierta funciona como una **pila LIFO** (Last In, First Out): los nuevos sucesores se insertan **al inicio** y se extrae siempre **del inicio**. Así siempre se expande el nodo más reciente (más profundo).

Analogía: Es como explorar una cueva con túneles. Eliges un túnel, vas hasta el fondo. Si no encuentras la salida, regresas al punto anterior (backtracking) y pruebas el siguiente túnel.

Propiedades

Característica	Resultado	Justificación
Compleitud	⚠ Solo con control de ciclos	Sin control puede caer en bucles infinitos y nunca terminar
Optimalidad	✗ No	No garantiza encontrar la solución de menor profundidad
Complejidad en tiempo	✗ $O(b^m)$	Puede explorar toda la rama más larga antes de encontrar la solución
Complejidad en espacio	✓ $O(b \times m)$	Solo guarda la rama activa (camino desde raíz hasta nodo actual)

Donde **m** es la profundidad máxima del árbol.

💡 **Pregunta de sesión:** ¿Puede un árbol ser infinito sin entrar en un ciclo?

Respuesta: Sí. Por ejemplo, el conjunto de los números naturales: desde 1 puedes ir a 2, desde 2 a 3... infinitamente. No hay ciclo, pero el árbol no tiene fondo. Para ese caso existe DFS con límite de profundidad.

¿Cuándo usar DFS? Cuando la memoria es muy limitada y se sospecha que la solución está profunda en el árbol. Su ventaja principal es el bajo consumo de memoria:

$$O(b \times m) \quad \text{vs} \quad O(b^d) \quad \text{de BFS.}$$

7. Búsqueda de Coste Uniforme — UCS (Uniform Cost Search)

Definición

Algoritmo que extiende BFS para manejar costes distintos entre acciones. En lugar de expandir por nivel, expande siempre el nodo con **menor coste acumulado** $g(n)$ desde el estado inicial.

La función de utilidad es: $f(n) = g(n)$ = coste real acumulado desde el inicio hasta el nodo n .

¿Por qué BFS no es suficiente con costes distintos?

BFS asume coste = 1 por acción. Si los costes varían, puede encontrar el camino de menos pasos pero no el de menor coste total.

Ejemplo del tema:

- p_1 = A-S-F-B: 3 pasos, coste = 450
- p_2 = A-S-R-P-B: 4 pasos, coste = 418

BFS elegiría p_1 (menos pasos), pero p_2 es más barato. UCS elegiría correctamente p_2 .

Mecánica

La cola abierta es una **cola de prioridad ordenada ascendentemente por $g(n)$** . En cada iteración se expande el nodo de menor coste acumulado, sin importar en qué nivel esté.

Analogía: UCS no le importa si tiene que caminar 10 km en 10 tramos pequeños; si eso cuesta menos que un solo tramo de 1 km que es carísimo, elegirá los 10 tramos. Siempre prioriza el camino más barato disponible, en cualquier nivel del árbol.

Propiedades

Característica	Resultado	Justificación
Complejidad	✓ Sí	Nunca descarta caminos; todos permanecen en la cola de prioridad
Optimalidad	✓ Sí	Siempre expande primero el camino de menor coste acumulado
Complejidad en tiempo	✗ Alta	Depende del coste de la solución óptima y el coste mínimo de operador
Complejidad en espacio	✗ Alta	Guarda todos los nodos pendientes con sus costes en la cola de prioridad

Condición necesaria para completitud y optimalidad: Todos los costes deben ser **positivos**. Si hubiera costes negativos o cero, el coste acumulado $g(n)$ podría no crecer y el algoritmo podría expandir el mismo nodo infinitamente.

8. Comparativa General de los Algoritmos

Característica	BFS	DFS	UCS
Complejitud	✓ Sí	⚠ Solo sin ciclos	✓ Sí
Optimalidad	✓ Solo si coste = 1	✗ No	✓ Sí
Complejidad tiempo	✗ $O(b^d)$	✗ $O(b^m)$	✗ Alta
Complejidad espacio	✗ $O(b^d)$	✓ $O(b \times m)$	✗ Alta
Estructura cola	FIFO (cola)	LIFO (pila)	Cola de prioridad por g(n)
Inserción de nodos	Al final	Al inicio	Ordenado por coste g(n)
Cuándo usar	Costes iguales, solución poco profunda	Memoria limitada, solución profunda	Costes distintos, necesidad de optimalidad

9. Estados Repetidos

Un problema común en todos los algoritmos: el mismo estado puede aparecer múltiples veces en el árbol, causando ciclos infinitos. Estrategias para manejarlo:

Estrategia	Descripción
Ignorarlo	Algunos algoritmos toleran duplicados por su orden de exploración
Evitar ciclos simples	No agregar el padre de un nodo a sus sucesores
Evitar ciclos generales	No agregar ningún ancestro de un nodo a sus sucesores

Evitar todos los repetidos	No agregar ningún nodo que ya esté en el árbol
----------------------------	--

10. Preguntas de Repaso (Estilo Examen)

Preguntas con respuesta

1. ¿Qué diferencia fundamental hay entre búsqueda en IA y aplicar Dijkstra?

En Dijkstra se conoce el grafo completo. En búsqueda en IA el grafo se desconoce — el agente lo va construyendo al explorar.

2. ¿Qué significa que un algoritmo sea completo?

Que garantiza encontrar la solución si esta existe.

3. ¿Qué significa que un algoritmo sea óptimo?

Que si hay varias soluciones, garantiza encontrar la de menor coste.

4. ¿Por qué BFS no es siempre óptima?

Porque asume coste = 1 por acción. Si los costes varían, el camino de menos pasos puede no ser el más barato.

5. ¿Por qué DFS no es completa en el caso general?

Porque puede caer en bucles infinitos si no se controlan los estados repetidos. Solo es completa si se garantiza la eliminación de estados repetidos dentro de una misma rama.

6. ¿Qué ventaja tiene DFS sobre BFS en memoria?

DFS solo guarda la rama activa: $O(b \times m)$ vs $O(b^d)$ de BFS. Mucho más eficiente cuando la solución está profunda.

7. ¿Por qué UCS requiere costes positivos para ser completo y óptimo?

Si hubiera costes cero o negativos, $g(n)$ podría no crecer y el algoritmo expandiría el mismo nodo indefinidamente.

8. Si la memoria es muy limitada y la solución está profunda, ¿qué algoritmo usas?

DFS, por su complejidad espacial $O(b \times m)$, mucho más eficiente que BFS o UCS.

9. Si los costes son distintos y necesitas la solución óptima, ¿qué algoritmo usas?

UCS, que ordena la exploración por coste acumulado $g(n)$ y garantiza optimalidad con costes positivos.

10. ¿Qué es el factor de ramificación b ?

El número promedio de sucesores que genera cada nodo al expandirse. Con $b = 10$ y $d =$

12 se tiene 10^{12} nodos — de ahí el costo exponencial de BFS.

11. Resumen Rápido para Repaso Express

Mnemónico para recordar las estructuras de cola:

- **BFS** → cola **FIFO** → explora por **niveles** → como **agua que inunda**
- **DFS** → pila **LIFO** → explora por **ramas** → como **explorar cuevas**
- **UCS** → cola de **prioridad** por $g(n)$ → explora por **coste acumulado** → como **GPS económico**

Reglas rápidas de decisión:

- ¿Costes iguales y solución poco profunda? → **BFS**
- ¿Memoria limitada y solución profunda? → **DFS**
- ¿Costes distintos y necesitas optimalidad? → **UCS**

	BFS	DFS	UCS
Completo	✓	⚠	✓
Óptimo	✓ (c=1)	✗	✓
Tiempo	$O(b^d)$	$O(b^m)$	Alto
Espacio	$O(b^d)$	$O(b \cdot m)$	Alto
Estructura	Cola FIFO	Pila LIFO	Cola prioridad